Solution: Maximum Depth (or Height) of Binary Tree (easy)

Problem Statement

Determine the depth (or height) of a binary tree, which refers to the number of nodes along the longest path from the root node to the farthest leaf node. If the tree is empty, the depth is 0.

Example:

1. - Input

   ```
       1
      / \
     2   3
    / \
   4   5
   ```

   Output: 3

   Explanation: The longest path is `1–>2–>4` or `1–>2–>5` with 3 nodes.

2. - **Input**:

     ```
     1
      \
       2
        \
         3
     ```

   - **Expected Output**: 3
   - **Justification**: There's only one path `1–>2–>3` with 3 nodes.

3. - **Input**:

     ```
         1
        / \
       2   3
      / \
     4   7
          \
           9
     ```

   - **Expected Output**: 4
   - **Justification**: The longest path is `1–>2–>7–>9` with 4 nodes.

Solution

What's the 'Depth'?

Picture a ladder. Each step is a level in our tree. The depth? It's just how many steps there are! If we have a 3-step ladder, our tree has a depth of 3.

To determine the deepest level of a binary tree, we'll use a depth-first search (DFS) approach. Begin at the root node and traverse down each branch, keeping track of the current depth. The depth increases by one each time we move to a child node. If a node is a leaf (no children), compare its depth with the current maximum depth and update if necessary. Recursively apply this process to each node's left and right children. This will cover all paths in the tree, allowing us to find and return the maximum depth encountered.

Breaking Down the Approach:

1. **Recursive Approach:** The depth of a binary tree can be computed recursively. The maximum depth of a tree is the maximum of the depths of its left and right subtrees plus one (the root node).
2. **Base Condition:** If the node is null, return 0. This ensures that we've reached the end of a branch or the tree is empty.
3. **Recursive Calculation:** For each node, compute the depth of its left subtree and the depth of its right subtree. The depth of the current node is 1 + the maximum of these two depths.
4. **Result:** For the root node, this recursive approach will provide the maximum depth of the entire tree.

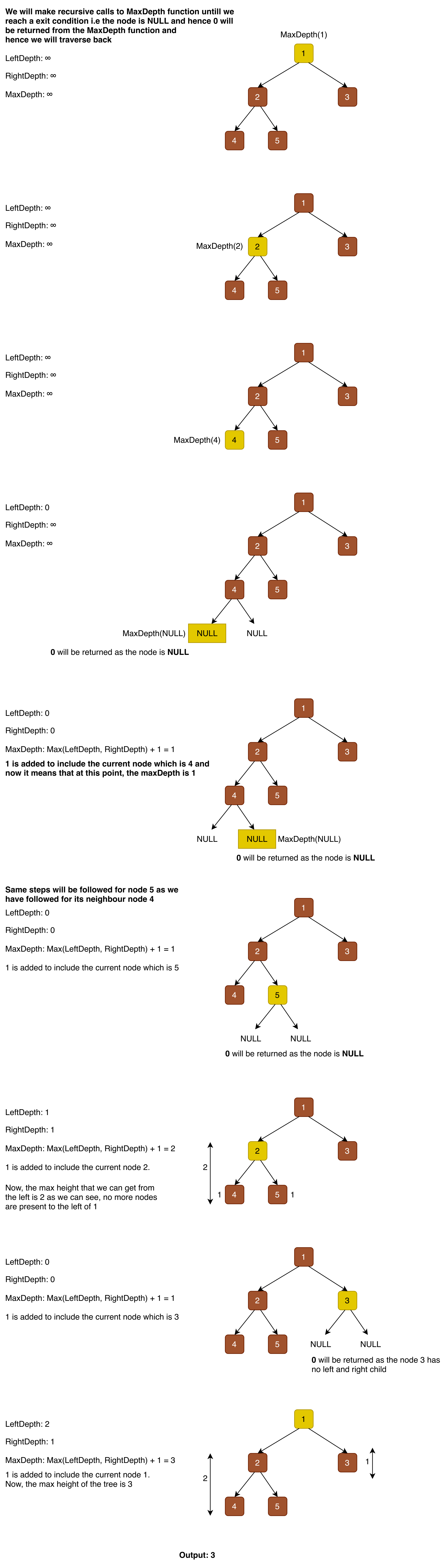
Algorithm Walkthrough:

Given Input:

```
    1
   / \
  2   3
 /
4
```

- Start with the root node which has value 1.
- The left subtree has a depth of:
  - 1 (for node 2) + max(depth of left subtree of node 2, depth of right subtree of node 2)
  - This is equal to 1 + 1 (for node 4) = 2.
- The right subtree of the root node has depth 1 (just node 3).
- So, the maximum depth of the tree is 1 (for the root node) + max(2, 1) = 3.

Here is the visual representation of the algorithm:



We will make recursive calls to MaxDepth function untill we reach a exit condition i.e the node is NULL and hence 0 will be returned from the MaxDepth function and hence we will traverse back

LeftDepth: ∞
RightDepth: ∞
MaxDepth: ∞

LeftDepth: ∞
RightDepth: ∞
MaxDepth: ∞

LeftDepth: ∞
RightDepth: ∞
MaxDepth: ∞

LeftDepth: 0
RightDepth: ∞
MaxDepth: ∞

0 will be returned as the node is **NULL**

LeftDepth: 0
RightDepth: 0
MaxDepth: Max(LeftDepth, RightDepth) + 1 = 1
1 is added to include the current node which is 4 and now it means that at this point, the maxDepth is 1

0 will be returned as the node is **NULL**

Same steps will be followed for node 5 as we have followed for its neighbour node 4
LeftDepth: 0
RightDepth: 0
MaxDepth: Max(LeftDepth, RightDepth) + 1 = 1
1 is added to include the current node which is 5

0 will be returned as the node is **NULL**

LeftDepth: 1
RightDepth: 1
MaxDepth: Max(LeftDepth, RightDepth) + 1 = 2
1 is added to include the current node 2.
Now, the max height that we can get from the left is 2 as we can see, no more nodes are present to the left of 1

LeftDepth: 0
RightDepth: 0
MaxDepth: Max(LeftDepth, RightDepth) + 1 = 1
1 is added to include the current node which is 3

0 will be returned as the node 3 has no left and right child

LeftDepth: 2
RightDepth: 1
MaxDepth: Max(LeftDepth, RightDepth) + 1 = 3
1 is added to include the current node 1.
Now, the max height of the tree is 3

Output: 3

Max. Depth of BT

Code

Here is the code for this algorithm:

```java
// Definition for a binary tree node.
// class TreeNode {
//     int val;
//     TreeNode left;
//     TreeNode right;
//     TreeNode(int x) { val = x; }
// }

public class Solution {

    // Method to compute the maximum depth of a binary tree
    public int maxDepth(TreeNode root) {
        // Base case: if node is null, return 0
        if (root == null) return 0;

        // Recursively calculate left subtree depth
        int leftDepth = maxDepth(root.left);

        // Recursively calculate right subtree depth
        int rightDepth = maxDepth(root.right);

```

Run Code Result Stdout

You must run your code first

Execute

Complexity Analysis

- Time Complexity: O(n) where n is the number of nodes in the tree, since we visit each node once.
- Space Complexity: O(h) where h is the height of the tree. This is because of the depth of the recursion stack, which is determined by the tree height.

← Previous
Maximum Depth (Or Height) Of Binary Tree (Easy)

Next →
Balanced Binary Tree (Easy)

☐ Mark as Completed